

AWS GenAI 課程 · 第 8 單元

高可用與自動擴展

盧建霖 • 2026-09-20

機器亮著綠燈，使用者看了一整晚轉圈圈。
今天要让服務掛一台也不會停。

Agenda

- 01 單點失敗與健康檢查
- 02 ALB 與 NLB：兩種負載平衡器
- 03 Auto Scaling：補機與加減台數
- 04 LLM 應用的擴展
- 05 實作：把常掛的單機改成不會掛

TODAY'S GOAL

手動終止任何一台機器，網站幾乎不受影響。

前三段搭骨架，第四段改成 LLM 服務的形狀，最後動手驗收。

先定義名詞，後面才好講

這六個詞今天會反覆出現，第一次出現就先給名字

單點失敗 **single point of failure**

這個元件一掛，整個服務就掛；只有一台機器就是單點。

負載平衡器 **load balancer**

站在使用者跟機器中間，把流量分給多台機器的那一台。

健康檢查 **health check**

定期對每台目標發一個請求，看它回不回、回什麼。

目標群組 **target group**

一組機器，加上它們共用的 **health check** 設定。

Auto Scaling 群組 **ASG**

一組被當成一個單位管理的 EC2，讓實際台數等於期望台數。

擴展策略 **scaling policy**

自動改「現在要幾台」這個數字的規則。

SINGLE POINT OF FAILURE

機器活著，不代表服務活著

- 一、只有一台就是單點失敗
- 二、**EC2** 看盒子，負載平衡器打服務
- 三、負載平衡器負責判斷，**Auto Scaling** 負責換機

服務死了機器沒死，預設檢查看不到

status check（狀態檢查）：AWS 自動做、免費，回答的是「這個盒子還在不在」

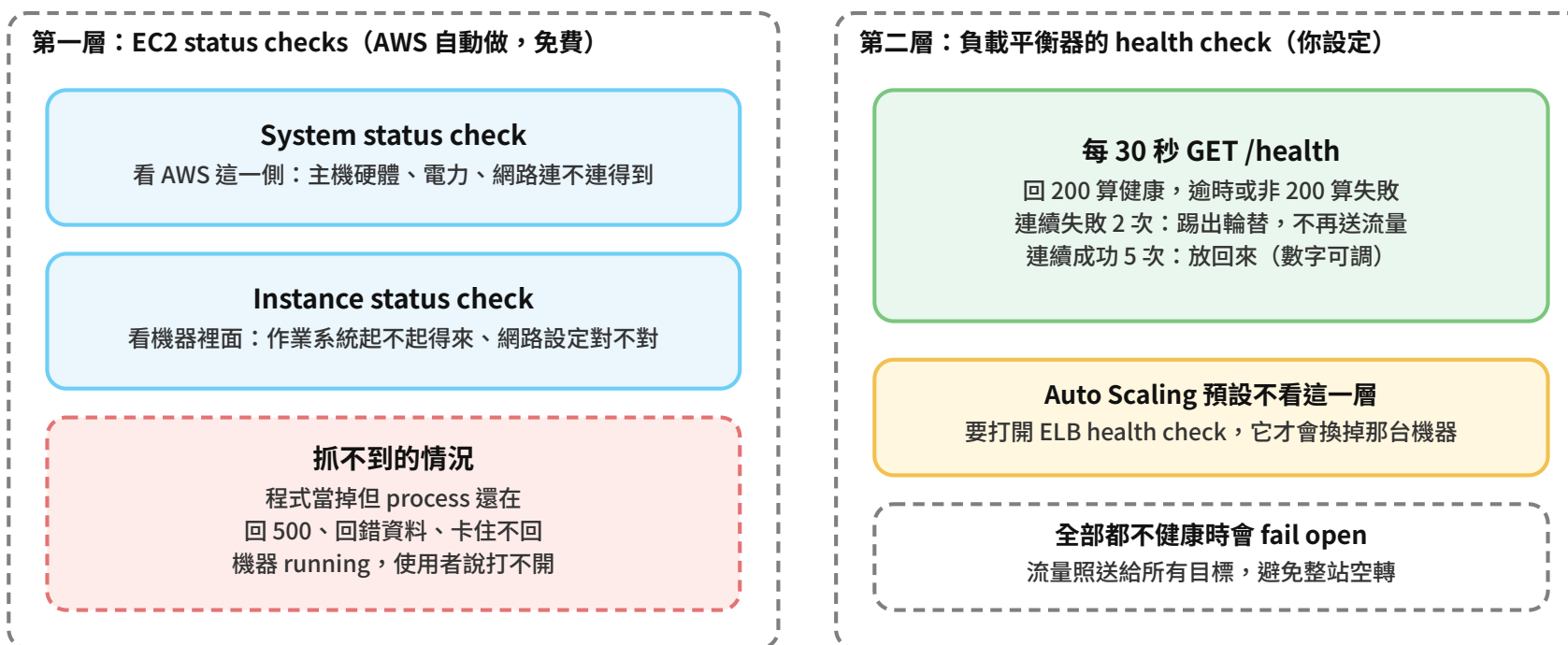
掛法	例子	誰看得到
AWS 那一側壞了	主機硬體、電力、網路連不到	EC2 的 system status check
機器裡面壞了	作業系統起不來、網路設定錯	EC2 的 instance status check
服務死了機器沒死	process 還在，但回 500 或卡住不回	system 跟 instance 都看不到

只有一台就是單點失敗；講師經驗裡最常見的是第三種掛法。

兩層健康檢查：一層看盒子，一層打服務

左邊是 EC2 自己的 status check，右邊是負載平衡器每 30 秒打一次 /health

兩層健康檢查：EC2 的檢查看「機器」，負載平衡器的檢查看「服務」



- 1 紅色虛線框：程式卡住、回 500，左邊兩個檢查都抓不到
- 2 綠框：要知道服務活不活，得有人定期打服務
- 3 黃框：Auto Scaling 預設不看右邊這一層，要自己打開

health check 看的是服務回不回得出答案

它不看機器開不開著；下面這幾個數字都可以調，先記邏輯

1 預設每 30 秒 GET 一次你指定的路徑

回 200 算過，逾時或回別的狀態碼算失敗。

2 連續失敗到門檻，就踢出輪替

不再送流量給它；連續成功到門檻再放回來。

3 全部都不健康時會 fail open

流量照送給所有機器；全部同時失敗多半是檢查設錯（講師的理解）。

負載平衡器踢掉壞機器，誰去換？

× 大家通常會想

負載平衡器判定不健康，那台機器就會被換掉。

踢出輪替只是不再送流量，機器還在原地。

✓ 實際上

Auto Scaling 預設只看 EC2 的 **status check**。

在群組上打開 ELB health check，它才會換掉負載平衡器判定不健康的機器。

負載平衡器做的三件事

多台機器對使用者看起來，像一台永遠在的機器

事情	白話	沒有它會怎樣
單一入口	使用者只認一個 DNS 名稱	每加一台就要改 DNS
分流到多個 AZ	每個啟用的 AZ 有一個節點	一個 AZ 掛，整站掛
只送給健康的	靠 health check 判斷	三分之一的請求打到死掉那台

它本身是 AWS 管的跨 AZ 服務，不會變成新的單點。

QUIZ 01 · CLF-C02

在設計 **AWS** 工作負載時，若希望即使發生元件故障仍能維持運作，哪一項是 **AWS** 的最佳實務？

- A** 每季執行一次災難復原測試
- B** 將主要元件放置在 us-east-1 區域 (Region)
- C** 設計為可自動容錯移轉至健康的資源
- D** 將工作負載設計為只需在單一 Amazon EC2 執行個體上執行

想 30 秒，等一下翻下一頁對答案。

在設計 **AWS** 工作負載時，若希望即使發生元件故障仍能維持運作，哪一項是 **AWS** 的最佳實務？

- ☐ A 每季執行一次災難復原測試
- ☐ B 將主要元件放置在 us-east-1 區域 (Region)
- ☒ C 設計為可自動容錯移轉至健康的資源
- ☐ D 將工作負載設計為只需在單一 Amazon EC2 執行個體上執行

C。元件故障仍能運作的最佳實務是自動容錯移轉到健康的資源：負載平衡器只送健康目標，Auto Scaling 群組補機；D 單一 EC2 就是單點失敗，B 放在特定 Region 跟容錯無關。

動手做：親手「打服務」，再看 **health check** 的預設值 (1/2)

小步	做什麼	做完應該看到
1	主控台上方導覽列點 CloudShell 圖示（或搜尋框打 CloudShell）	下方或新視窗開出一個終端機，命令列上方寫著區域名稱
2	貼上 <code>curl -s -o /dev/null -w "%{http_code}\n" https://aws.amazon.com/</code> 按 Enter	只印出一個數字 200，health check 看到這個就記一次成功
3	把網址換成 <code>https://aws.amazon.com/this-page-does-not-exist-123/</code> 再跑一次	印出 404；伺服器活得好好的，但這個路徑的「服務」不存在，health check 會記一次失敗

動手做：親手「打服務」，再看 health check 的預設值 (2/2)

小步	做什麼	做完應該看到
4	回主控台：EC2 → 左側 Load Balancing 底下的 Target Groups → Create target group，往下捲到 Health checks 區	Health check protocol 是 HTTP，Health check path 預設是 /
5	展開 Advanced health check settings	Healthy threshold 5、Unhealthy threshold 2、Timeout 5 秒、Interval 30 秒、Success codes 200
6	不按 Next，直接按 Cancel 離開	Target Groups 列表沒有多出任何東西

第 2、3 步就是負載平衡器每 30 秒在做的事，它只看回應碼，不看機器開不開著，這就是本課「兩層健康檢查」那一段講的「要問服務就得打服務」。

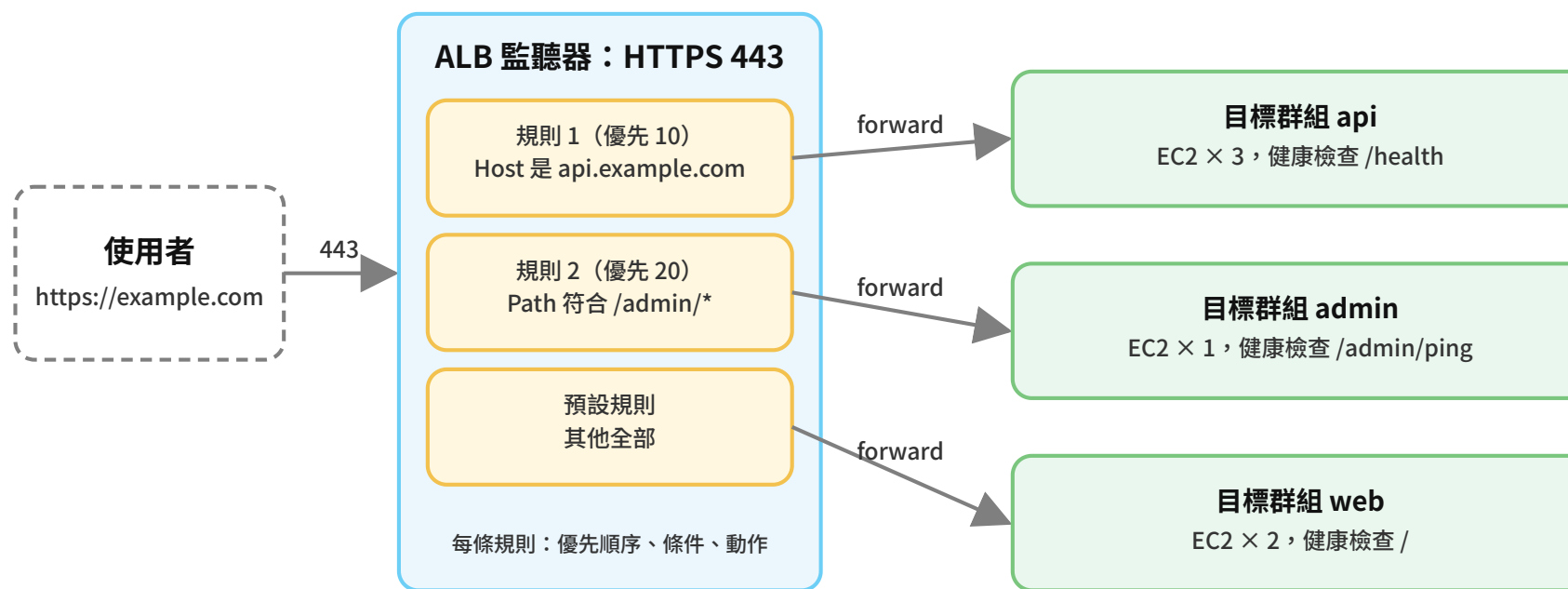
ALB 看信的內容，NLB 只看信封

- 一、第七層看得懂 **HTTP**，網址、主機名、**header** 都能分流
- 二、第四層只看 **IP** 跟埠，換來速度與固定 **IP**
- 三、高併發、固定 **IP**、非 **HTTP** 選 **NLB**，其他預設 **ALB**

ALB 四個零件：監聽器、規則、目標群組、目標

ALB (Application Load Balancer)：看得懂 HTTP 的負載平衡器，在 OSI 第七層（應用層）工作

ALB 看得懂 HTTP：一個監聽器、幾條規則、每條規則把請求送去一個目標群組



- 1 藍色大框是監聽器 (listener)，聽 443 埠
- 2 黃色小框是規則，數字小的先比，比中就停；一定有一條預設規則
- 3 每條規則 forward 到一個目標群組，群組各有自己的 health check

規則 = 條件加動作，左右自由組合

條件（看什麼）	例子	動作（做什麼）
host-header 主機名	api.example.com	forward ：轉給某個目標群組
path-pattern 路徑	/admin/*	redirect ：轉址，例如 http 轉 https
http-header 任一 header	X-Client: mobile	fixed-response ：直接回一段文字
http-request-method 方法	POST	authenticate ：先驗身分再轉
query-string 查詢字串	version=beta	

左右不是同列配對；條件還有 **source-ip**，一條規則最多**五個比對值**。

專題會用到的兩條規則

以前寫在 nginx 裡的事，現在寫在 AWS 的設定裡

1 `/api/*` 轉給 agent 後端的目標群組

其他請求轉給前端那組機器。

2 80 埠的監聽器只放一條 redirect 到 443

前提是 443 已經建好 HTTPS 監聽器跟憑證；不用改程式就強制 HTTPS。

3 機器換了，規則還在

規則寫在 ALB 的設定裡，不跟著任何一台機器。

看得懂 HTTP，附送的不只分流

功能	白話	對你的意義
HTTP/2 與 WebSocket	一條連線多個請求；升級成雙向長連線	聊天介面可以走 WebSocket
Lambda 當目標	目標不一定是機器	serverless 後端也能掛同一個 ALB
AWS WAF (Web Application Firewall)	掛在 ALB 前面	擋 SQL 注入這類第七層攻擊
閒置逾時	一條連線多久沒資料就切，預設 60 秒	LLM 串流會踩到這個坑

連線升級成 WebSocket 之後，監聽器規則跟 WAF 不再套用。

QUIZ 02 · CLF-C02

一個電子商務公司已經在 **Amazon EC2** 執行個體上部署了新的網頁應用程式。公司希望將進來的 **HTTP** 流量均勻地分配到所有運行中的執行個體上。哪一個 **AWS** 服務或資源可以滿足這些需求？

- ☐ A Amazon EC2 自動擴展 (Auto Scaling)
- ☐ B 應用程式負載平衡器 (Application Load Balancer)
- ☐ C 閘道負載平衡器 (Gateway Load Balancer)
- ☐ D 網路負載平衡器 (Network Load Balancer)

想 30 秒，等一下翻下一頁對答案。

QUIZ 02 · CLF-C02 · ANSWER

一個電子商務公司已經在 **Amazon EC2** 執行個體上部署了新的網頁應用程式。公司希望將進來的 **HTTP** 流量均勻地分配到所有運行中的執行個體上。哪一個 **AWS** 服務或資源可以滿足這些需求？

- ☐ A Amazon EC2 自動擴展 (Auto Scaling)
- ☒ B 應用程式負載平衡器 (Application Load Balancer)
- ☐ C 閘道負載平衡器 (Gateway Load Balancer)
- ☐ D 網路負載平衡器 (Network Load Balancer)

B。「HTTP 流量、均勻分配到多台 EC2」選 Application Load Balancer；A Auto Scaling 管台數不分流量，D NLB 也能分但題目講 HTTP 就選第七層的。

動手做：拆開一個 HTTP 請求，找出規則能看的每一格（1/2）

小步	做什麼	做完應該看到
1	主控台上方導覽列點 CloudShell 圖示	開出一個終端機
2	貼上 <code>`curl -sv --http1.1 -o /dev/null https://checkip.amazonaws.com/ 2>&1\`</code>	<code>grep '^> '`</code> 四行以 > 開頭：GET / HTTP/1.1、Host: checkip.amazonaws.com、User-Agent: curl/...、Accept: */*
3	對著條件表把四行配對：GET 對 http-request-method、/ 對 path-pattern、Host 對 host-header、User-Agent 對 http-header	條件表六種條件裡，四種直接出現在這幾行

動手做：拆開一個 HTTP 請求，找出規則能看的每一格（2/2）

小步	做什麼	做完應該看到
4	貼上 <code>curl -s https://checkip.amazonaws.com/</code>	印出一個 IP， 就是這個請求的 來源 IP， <code>source-ip</code> 條件 看的是它
5	貼上 <code>`curl -sI http://aws.amazon.com/`</code> <code>`</code>	<code>grep -i -E '^location'</code> <code>'^HTTP\`</code> HTTP/1.1 301 Moved Permanently 跟 Location: <code>https://aws.amazon.com/</code> ，這就是 redirect 動作把 http 轉成 https 的樣子

第 2 到 4 步看到的每一格，就是本課「規則的條件與動作」那張表左欄能比對的東西；第四層負載平衡器只看得到第 4 步那個 IP 跟埠，看不到其他幾行。

NLB 什麼都不看，換速度跟固定 IP

NLB (Network Load Balancer)：只看 IP 跟埠、不拆內容，在 OSI 第四層（傳輸層）工作

面向	ALB	NLB
層級	第七層，拆開 HTTP 看內容	第四層，只看 IP 跟埠
協定	HTTP、HTTPS、WebSocket、gRPC	TCP、UDP、TLS、QUIC
位址	只給 DNS 名稱，IP 會變	每個 AZ 一個固定 IP
分流依據	規則：主機名、路徑、header	flow hash：同一條連線送同一台
附送	轉址、WAF、Lambda 目標、身分驗證	依目標類型可保留來源 IP

效能：NLB 每秒百萬級請求；延遲比 ALB 低是講師經驗。

不拆內容所以快，每個 AZ 一個固定 IP

左邊 ALB、右邊 NLB，各三格：能做的事、代價、典型用途

ALB 拆開信看內容再決定送哪；NLB 只看信封，所以快、能給固定 IP、不限 HTTP

ALB：第七層（應用層）

拆開 HTTP 看內容

主機名、路徑、header、方法、查詢字串
依內容轉發、轉址、直接回應、先驗證身分
支援 HTTP/2、WebSocket、Lambda 當目標

代價

要看內容就慢一點；IP 會變，只給你 DNS 名稱

典型用途

網站、REST API、微服務依路徑分流、掛 WAF

NLB：第四層（傳輸層）

只看信封：IP 跟埠

TCP、UDP、TLS、QUIC 都能轉
同一條連線固定送同一台（flow hash）
每秒百萬級請求

特權

每個 AZ 一個固定 IP，對外型可綁 Elastic IP

典型用途

遊戲、IoT、資料庫代理、客戶防火牆只放行固定 IP

- 1 NLB 黃框「每個 AZ 一個固定 IP」是兩者最常被考的差別
- 2 對外型的 NLB 還可以每個子網路綁一個 Elastic IP
- 3 NLB 的跨 AZ 分流預設關：一邊 1 台一邊 5 台，1 台那邊扛一半流量

三個訊號詞任一個出現，就選 NLB

看到	選	為什麼
極高併發、極低延遲、每秒百萬請求	NLB	第四層不拆內容
客戶要白名單、每個 AZ 要固定 IP	NLB	ALB 只給 DNS 名稱
非 HTTP：遊戲、IoT、自訂 TCP 協定	NLB	ALB 只懂 HTTP
看網址分流、轉址、WAF、WebSocket	ALB	第七層才看得懂

兩種都要：NLB 在前給固定 IP，ALB 當它的目標繼續分流（講師補充）。

Region 內用 ELB，跨 Region 靠另外兩個

ALB 跟 NLB 都只在一個 Region 裡分流；CLF 幾乎不直接考 NLB，考的是它的近親

1 Global Accelerator

AWS 在邊緣給兩個全球固定 IP，把使用者導到最近或最健康的 Region。

2 Route 53

用 DNS 回覆把使用者導到別的 Region。

3 NLB 解的是一個 Region 內的固定 IP

專題的 agent API 是 HTTP，多半用不到 NLB。

QUIZ 03 · CLF-C02

以下哪一項 **AWS** 服務可協助在多區域 (**multi-Region**) 與多可用區 (**Multi-AZ**) 架構中，提供具備快速容錯移轉 (**fast failover**) 的高可用性應用程式？

A

AWS WAF

B

AWS Global Accelerator

C

AWS Shield

D

AWS Direct Connect

想 30 秒，等一下翻下一頁對答案。

QUIZ 03 · CLF-C02 · ANSWER

以下哪一項 **AWS** 服務可協助在多區域 (**multi-Region**) 與多可用區 (**Multi-AZ**) 架構中，提供具備快速容錯移轉 (**fast failover**) 的高可用性應用程式？

A AWS WAF

B AWS Global Accelerator

C AWS Shield

D AWS Direct Connect

B。「多 Region 加多 AZ、快速容錯移轉」選 Global Accelerator，它給兩個全球固定 IP 並把使用者導到健康的 Region；WAF 跟 Shield 是防護，Direct Connect 是專線。

動手做：並排打開 **ALB** 跟 **NLB** 的建立頁，找出兩個差別

小步	做什麼	做完應該看到
1	EC2 → 左側 Load Balancing 底下的 Load Balancers → Create load balancer	幾張卡片並排，其中有 Application Load Balancer 跟 Network Load Balancer
2	Application Load Balancer 那張按 Create，捲到 Listeners and routing，點開 Protocol 下拉選單	預設是 HTTP、Port 80，選單只有 HTTP 跟 HTTPS
3	按瀏覽器上一頁回到卡片頁，Network Load Balancer 那張按 Create，捲到 Listeners and routing，點開 Protocol 下拉選單	預設是 TCP、Port 80，選單是 TCP、TLS、UDP、TCP_UDP 這類第四層協定，沒有 HTTP
4	同一頁 Scheme 保持 Internet-facing，Network mapping 的 VPC 選預設，勾一個 AZ（例如 ap-northeast-1a）	那個 AZ 底下除了子網路，還多一個 IPv4 位址的設定，可以改成用你自己的 Elastic IP
5	頁面最下方按 Cancel，不按 Create load balancer	Load Balancers 列表沒有多出任何東西

第 2、3 步看到的是本課「一張表看差別」的協定那一列，第 4 步看到的是位址那一列，NLB 能每個 AZ 綁一個固定 IP，ALB 的建立頁沒有這一格。

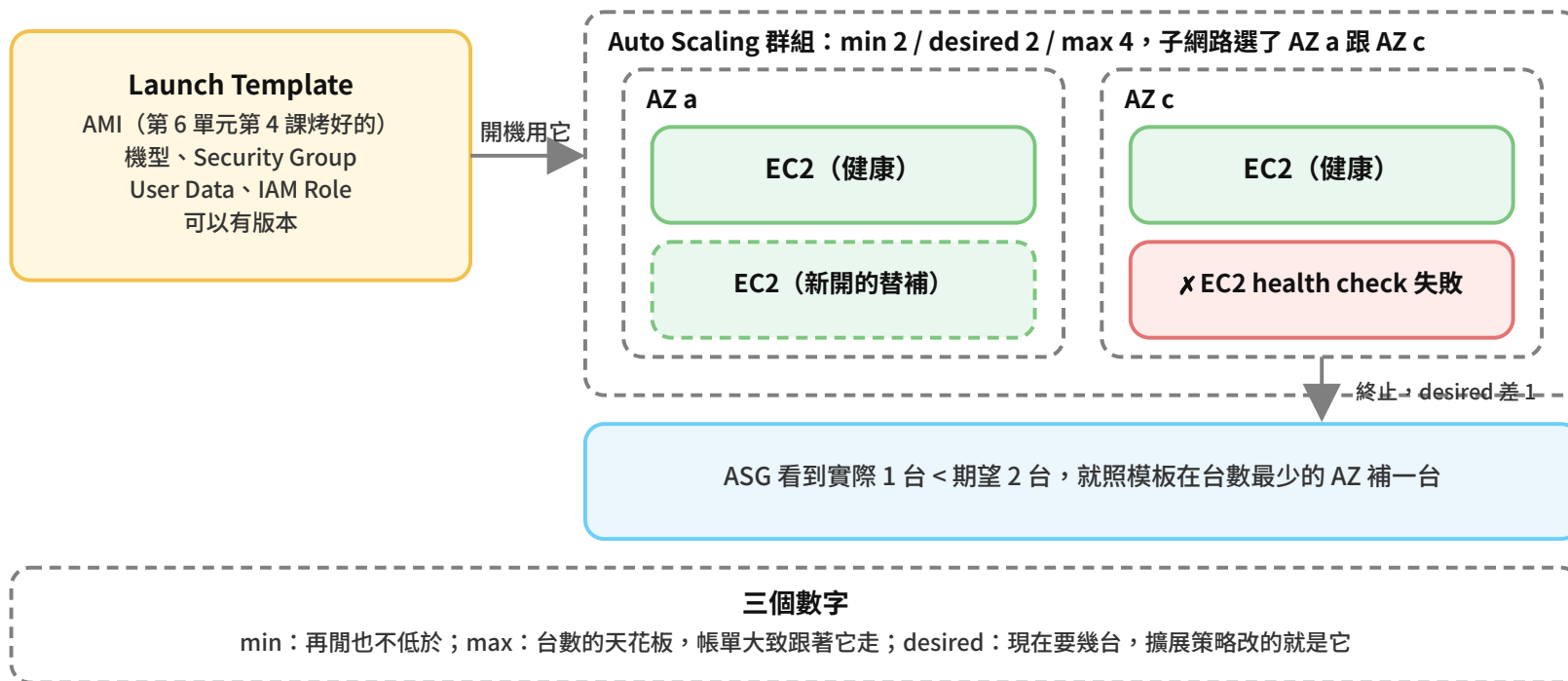
把「應該有幾台」寫成數字，交給服務盯

- 一、**ASG** 只做一件事：讓實際台數等於期望台數
- 二、要打開 **ELB health check**，才看得到服務死掉
- 三、擴展指標一定要是「每台有多忙」，不能是總量

一張模板加三個數字，掛一台就補一台

Launch Template（啟動模板）：寫好 AMI、機型、Security Group、User Data、IAM Role，開每一台都照它

Auto Scaling 群組只做一件事：讓實際台數等於期望台數；模板決定每一台長什麼樣



- 1 左邊黃框是 **Launch Template**，箭頭指向中間的群組，兩個 AZ 各有機器
- 2 紅色那台 **health check** 失敗被終止，下面藍框是群組的反應
- 3 最底下灰框：min、max、desired 三個數字各管什麼

ASG 不修機器，它換機器

grace period（寬限期）：剛開機那幾分鐘，ASG 先不把檢查失敗當成換機理由

步驟	發生什麼
1 監控	預設用 EC2 status check 判斷；打開 ELB health check 後也聽負載平衡器的
2 判定	連續失敗就標成 Unhealthy；新機器有一段寬限期
3 終止	終止那台；有 connection draining 就先等進行中的請求做完
4 補位	實際台數低於 desired，照模板在機器最少的 AZ 補一台

少的原因是掛了、你手動終止、還是 AZ 整個不見，它都照補。

三件事 **ASG** 不會替你做

建群組時自己要設對，機器上的東西自己要搬走

1 打開 ELB health check

預設只看 EC2 status check，程式死了它不知道。

2 選兩個 AZ 的子網路

AZ 由你選的子網路決定；只選一個 AZ，它就只開在那裡。

3 機器要無狀態

檔案放 S3 或 EFS，session 放資料庫或快取；機器隨時會被換掉。

機器是牛，不是寵物。

On-Demand 保底，Spot 接尖峰

前提是無狀態；列的機型越多，Spot 越不容易缺貨

設定	白話	例子
On-Demand 基底	這幾台一定是 On-Demand，保底	2 台
基底以上的比例	超過基底的機器幾成 On-Demand、幾成 Spot	30% On-Demand、70% Spot
多種機型	列幾個能接受的機型，Spot 缺貨換一種	m7g.large、m6g.large、c7g.large

Spot 被收回，ASG 當成一台掛了照樣補；專題先用兩台 On-Demand 跑通。

QUIZ 04 · CLF-C02

解決方案架構師需要維護一個 **Amazon EC2** 實例群組，以便任何受損的實例都能被新的實例取代。解決方案架構師應該使用哪一個 **AWS** 服務？

A Amazon Elastic Container Service (Amazon ECS)

B Amazon GuardDuty

C AWS Shield

D AWS Auto Scaling

想 30 秒，等一下翻下一頁對答案。

QUIZ 04 · CLF-C02 · ANSWER

解決方案架構師需要維護一個 **Amazon EC2** 實例群組，以便任何受損的實例都能被新的實例取代。解決方案架構師應該使用哪一個 **AWS** 服務？

A Amazon Elastic Container Service (Amazon ECS)

B Amazon GuardDuty

C AWS Shield

D AWS Auto Scaling

D。「受損的實例自動被新實例取代」是 Auto Scaling 群組的自癒：監控、判定、終止、補位；ECS 跑容器，GuardDuty 跟 Shield 是安全服務。

動手做：走一遍 **ASG** 建立精靈，看三個數字互相卡住 (1/2)

小步	做什麼	做完應該看到
1	EC2 → 左側 Instances 底下的 Launch Templates → Create launch template：名稱 handson-It，AMI 在 Quick Start 選 Amazon Linux，Instance type 選 t3.micro，其他不填 → Create launch template	成功頁面，Launch Templates 列表有 handson-It
2	EC2 → Auto Scaling Groups → Create Auto Scaling group：名稱 handson-asg，Launch template 選 handson-It → Next	進到 Choose instance launch options 頁
3	VPC 選預設，Availability Zones and subnets 選 ap-northeast-1a 跟 ap-northeast-1c 各一個子網路 → Next，下一頁 Integrate with other services 不動 → Next	進到 Configure group size and scaling 頁

動手做：走一遍 **ASG** 建立精靈，看三個數字互相卡住 (2/2)

小步	做什麼	做完應該看到
4	Desired capacity 填 2；Scaling limits 的 Min desired capacity 填 2、Max desired capacity 填 4	三個數字是 2、2、4，就是大 Lab 要填的那組
5	把 Desired capacity 改成 5	Max desired capacity 自動被抬成 5：desired 不能超出 min 到 max 的範圍
6	按 Cancel 離開精靈；再到 Launch Templates 勾 handson-It → Actions → Delete template，輸入 Delete 確認	Auto Scaling Groups 列表沒有 handson-asg，Launch Templates 列表沒有 handson-It

第 4、5 步就是本課「三個數字跟一張模板」那段講的邊界，desired 永遠被夾在 min 跟 max 之間，ASG 做的事就是讓實際台數追上 desired。

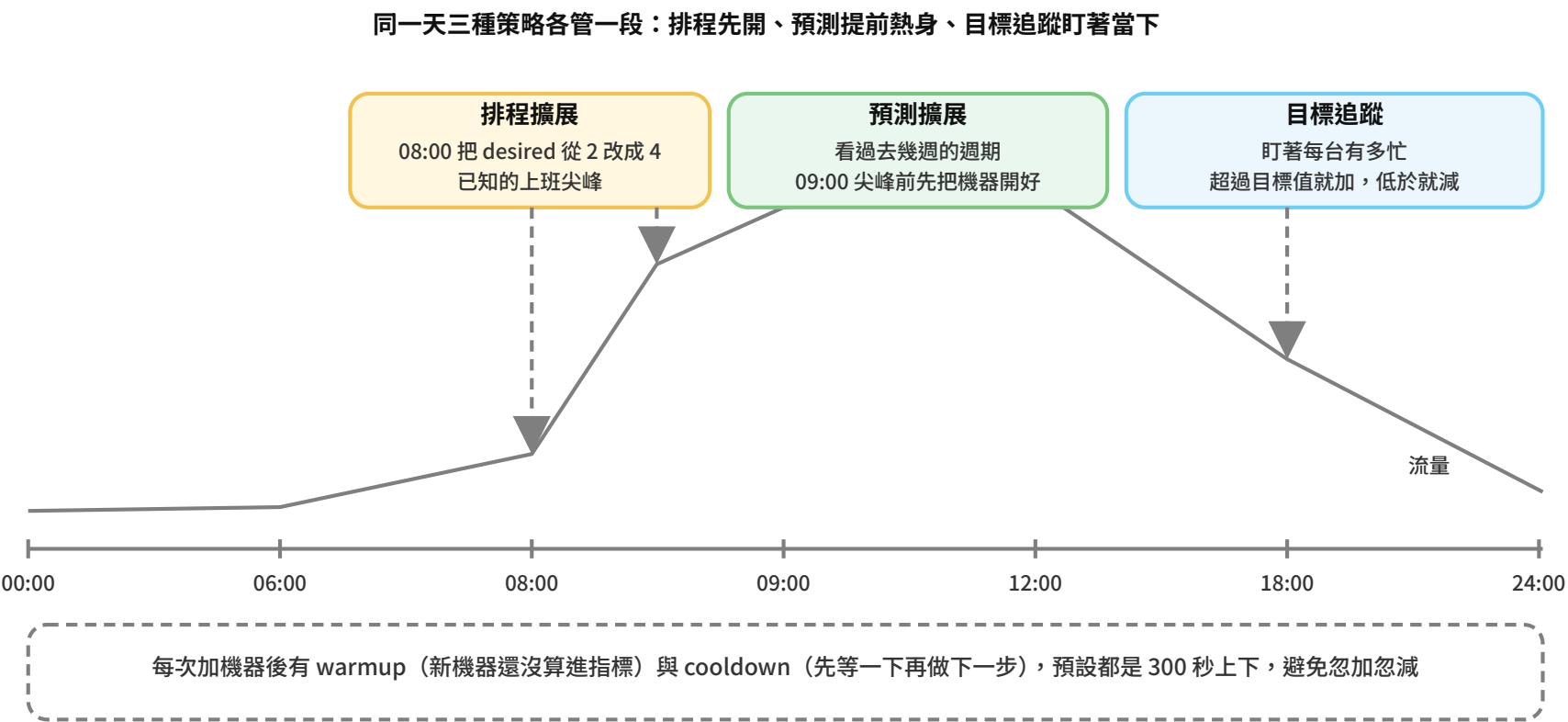
四種策略：反應現在，或提前動手

策略	白話	誰觸發	適合
目標追蹤 target tracking	像恆溫器：每台 CPU 維持 50%	CloudWatch 指標偏離目標	大多數情況的預設
步進與簡單 step / simple	CPU 過 60 加 2 台，過 80 加 4 台	你設的 CloudWatch 警報	要精細控制每一階
排程 scheduled	平日 08:00 改成 4，20:00 改回 2	時間，cron 語法	已知的尖峰
預測 predictive	每天九點都衝高，八點五十先開	機器學習對歷史流量的預測	有週期又開機慢的服務

目標追蹤盯著現在；排程跟預測在尖峰之前動手；四種可以疊著用。

尖峰前有機器，尖峰中不缺，尖峰後不浪費

灰線是一天的流量，九點衝高、傍晚降下；上面三個框是三種策略各在什麼時候動手



- 1 黃色排程八點抬高 **desired**，綠色預測在尖峰前開好，藍色目標追蹤整天盯著
- 2 **warmup** (暖機期)：新機器這段時間的指標不算進群組平均
- 3 **cooldown** (冷卻期)：簡單擴展做完一次動作先等，預設 300 秒

先問尖峰能不能預知

能就排程或預測，不能就目標追蹤

1 預知不到：目標追蹤，不用算要加幾台

設每台 CPU 維持 50%，兩台各 90% 時它自己算出要四台。

2 已知的尖峰：排程，注意時區

cron 沒指定時區就是 UTC，台灣 06:30 是 UTC 前一天 22:30。

3 有週期又開機慢：預測

需要歷史資料；新上線、活動促銷這種一次性尖峰它看不出來。

題庫說的彈性（Elasticity），就是 **desired** 會自己動。

指標要反映「每台有多忙」，不能是總量

規則	白話	例子
要反映每台有多忙	台數加倍，指標要跟著掉一半	平均 CPU、每台請求數
不能用總量	總請求數不會因為多開機器變少	負載平衡器的總請求數
解析度一分鐘以內	五分鐘一筆反應太慢（官方建議）	EC2 開 detailed monitoring
排隊型工作看每台積壓量	佇列長度除以在線台數	SQS 佇列訊息數 ÷ 在線台數

CPU 只有真的是瓶頸時才準；忙在等 GPU、等下游，CPU 可能只有 20%。

QUIZ 05 · CLF-C02

哪一個架構概念描述了能夠按需部署資源，並在不再需要時釋放資源的能力？

A

高可用性 (High availability)

B

解耦架構 (Decoupled architecture)

C

韌性 (Resilience)

D

彈性 (Elasticity)

想 30 秒，等一下翻下一頁對答案。

QUIZ 05 · CLF-C02 · ANSWER

哪一個架構概念描述了能夠按需部署資源，並在不再需要時釋放資源的能力？

A 高可用性 (High availability)

B 解耦架構 (Decoupled architecture)

C 韌性 (Resilience)

D 彈性 (Elasticity)

D。「按需部署、不需要時釋放」是彈性的定義句；高可用著重減少停機，韌性講從故障恢復，解耦著重降低元件相依與故障的連帶影響。

動手做：打開目標追蹤的指標選單，確認裡面沒有「總量」(1/2)

小步	做什麼	做完應該看到
1	EC2 → Launch Templates → Create launch template：名稱 handson-It，AMI 在 Quick Start 選 Amazon Linux，Instance type 選 t3.micro，其他不填 → Create launch template	Launch Templates 列表有 handson-It
2	EC2 → Auto Scaling Groups → Create Auto Scaling group：名稱 handson-asg，Launch template 選 handson-It → Next；VPC 選預設、子網路選 ap-northeast-1a 跟 1c → Next → Next	進到 Configure group size and scaling 頁
3	Automatic scaling 選 Target tracking scaling policy	出現 Metric type、Target value、Instance warmup 三個欄位

動手做：打開目標追蹤的指標選單，確認裡面沒有「總量」(2/2)

小步	做什麼	做完應該看到
4	點開 Metric type 下拉選單	四個選項：群組平均 CPU、平均網路進、平均網路出、Application Load Balancer request count per target；每一個都是「平均每台」，沒有「總請求數」
5	看 Instance warmup 那一格	一個以秒為單位的數字，就是本課講的 warmup：新機器這段時間的指標不算進平均
6	按 Cancel 離開精靈；再到 Launch Templates 勾 handson-It → Actions → Delete template，輸入 Delete 確認	Auto Scaling Groups 列表沒有 handson-asg，Launch Templates 列表沒有 handson-It

第 4 步那個選單本身就是本課「選指標的規則」第一條的證據，AWS 內建給你的四個指標全是「每台有多忙」，總量根本不在選項裡。

前面的做法套在 **LLM** 服務，三個地方會壞

- 一、指標看每台同時請求數，不看 **CPU**
- 二、連線要撐得過長回應：串流，調大閒置逾時
- 三、健康檢查要真的問模型一句

三個地方都不會報錯，只會慢慢爛

LLM 服務一個請求跑幾秒到幾十秒，一路吐字；前面的做法是為幾十毫秒回一個 JSON 的網站設計的

LLM 服務套 A04 會壞在三個地方：指標、逾時、健康檢查

指標：CPU 不準

推論卡在 GPU 跟排隊，CPU 可能只有 20%
用 CPU 目標追蹤永遠不會擴
改看「每台同時處理幾個請求」
或「每台排隊幾個」（SQS 那套）

逾時：長連線會被切

ALB 閒置逾時預設 60 秒
串流回答 token 一直在流，連線活著
第一個 token 等超過 60 秒，就斷了
調高逾時，或先送一個 byte 佔線

健康檢查：要真的問一句

/health 回 200 只證明 web 框架活著
模型沒載好、GPU 記憶體爆了，它照樣 200
健康檢查跑一個極小的推論
或至少確認模型已載入、下游可達

共同原因：一般 API 幾十毫秒回完、瓶頸在 CPU；LLM 回答要幾秒到幾十秒、瓶頸在 GPU 與併發（時間數字是講師經驗）

專題怎麼辦

用 Bedrock 時模型那側 AWS 扛；你的 agent 服務仍要 ALB 加 ASG、調高閒置逾時、健康檢查確認能連到 Bedrock

- 1 紅框指標：CPU 不忙，GPU 跟排隊滿了
- 2 黃框逾時：長連線被 60 秒閒置逾時切掉
- 3 綠框健康檢查：/health 回 200，但模型根本沒載好

每一列都是前面某個設定要改

面向	一般 Web API	LLM 推論服務
一個請求多久	幾十毫秒（講師經驗）	幾秒到幾十秒（講師經驗）
瓶頸在哪	CPU	GPU 記憶體、同時處理的請求數
擴展指標	平均 CPU、每台請求數	每台同時請求數、每台排隊數
新機器多久能用	秒級（講師經驗）	分鐘級，要載幾十 GB 權重（講師經驗）
健康檢查	/health 回 200	跑極小的推論，或確認模型已載入

開機慢的服務，正是官方說**預測擴展**最適合的對象。

LLM 服務也用 CPU 50% 做目標追蹤？

× 大家通常會想

平均 CPU 是內建指標，拿來做目標追蹤最省事。

機器已經滿載，CPU 可能只有 20%，策略以為很閒，擴不起來。

✓ 實際上

改看每台同時處理的請求數，或每台排隊的請求數。

自架 LLM 推論的忙多半在 GPU 跟排隊（講師經驗）；這兩個都是自訂指標。

閒置逾時計的是「多久沒資料」

串流 (streaming)：模型每生成一段就先送出，不等整段答完

1 token 持續在流，就不會被切

會被切的是超過 60 秒沒有 byte 的空窗：第一個 token 等太久、非串流一次回。

2 官方解法兩條

每個閒置逾時週期內至少送一個 byte；把閒置逾時調大。

3 服務自己的 keep-alive 逾時要比 ALB 的長

不然服務先關連線，ALB 又把下一個請求送進來，使用者拿到 502。

Bedrock 的串流 API 是 ConverseStream：收到 delta 就往前送，連線一直有資料。

用 **Bedrock**，你只管自己的 **agent** 服務

設定	沒改的樣子	專題改成
閒置逾時	預設 60 秒	調到幾分鐘
回應方式	非串流，一次回	一律串流
健康檢查	/health 固定回 200	確認能連到 Bedrock
擴展指標	平均 CPU	每台同時請求數

模型那一側的擴展是 AWS 的事，你不用管 GPU；agent 服務還是 ALB 加 ASG。

“

LLM 服務是另一種形狀的服務：
看併發不看 CPU、連線要撐得住、
健康檢查要真的問一句。

—— LLM 應用的擴展 ——

QUIZ 06 · CLF-C02

一家公司希望在 **AWS** 上部署非容器化的 **Java** 網頁應用程式。公司希望使用受管服務快速部署應用程式，並希望該服務能自動配置容量、負載平衡、擴展以及監控應用程式健康狀態。哪一個 **AWS** 服務能滿足這些需求？

- ☐ A Amazon Elastic Container Service (Amazon ECS)
- ☐ B AWS Lambda
- ☐ C Amazon Elastic Kubernetes Service (Amazon EKS)
- ☐ D AWS Elastic Beanstalk

想 30 秒，等一下翻下一頁對答案。

QUIZ 06 · CLF-C02 · ANSWER

一家公司希望在 **AWS** 上部署非容器化的 **Java** 網頁應用程式。公司希望使用受管服務快速部署應用程式，並希望該服務能自動配置容量、負載平衡、擴展以及監控應用程式健康狀態。哪一個 **AWS** 服務能滿足這些需求？

- ☐ A Amazon Elastic Container Service (Amazon ECS)
- ☐ B AWS Lambda
- ☐ C Amazon Elastic Kubernetes Service (Amazon EKS)
- ☒ D AWS Elastic Beanstalk

D。自動配置容量、負載平衡、擴展、健康監控，正是 Elastic Beanstalk 的整套，它把這個單元打包成一個服務；ECS 跟 EKS 是容器平台，題目說非容器化；Lambda 是函式執行環境。

動手做：讓 curl 扮演閒置逾時 3 秒的 ALB，看誰被切（1/2）

小步	做什麼	做完應該看到
1	主控台上方導覽列點 CloudShell 圖示	開出一個終端機
2	貼上 curl -s -o /dev/null -w "%{http_code} 總共 %{time_total} 秒\n" https://httpbin.org/delay/6	200 總共約 6 到 8 秒：沒有人切它的時候，悶 6 秒也拿得到回答

動手做：讓 **curl** 扮演閒置逾時 3 秒的 **ALB**，看誰被切 (2/2)

小步	做什麼	做完應該看到
3	貼上 <code>curl -sS -o /dev/null --speed-time 3 --speed-limit 1 https://httpbin.org/delay/6</code>	約 3 秒後印出 <code>curl: (28) Operation too slow. Less than 1 bytes/sec transferred the last 3 seconds</code> ，被切了
4	貼上 <code>curl -sS -o /dev/null --speed-time 3 --speed-limit 1 -w "%{http_code} 首個 byte %{time_starttransfer} 秒，總共 %{time_total} 秒\n" "https://httpbin.org/drip?duration=6&numbytes=60&delay=0"</code>	200，首個 byte 約 1 秒內、總共約 6 到 8 秒：總時間遠超過 3 秒，但一直有資料在流，沒被切

第 3、4 步的回答都要 6 秒以上才送完，一個被切一個沒事，差別只在「中間有沒有空窗」，這就是本課「串流與逾時」那段講的，閒置逾時計的是多久沒資料，不是總共多久；檢查點 Q2 的 60 秒跟 90 秒是同一個道理。

零程式碼，把常掛的單機改成不會掛

- 一、講師給一支會自己崩的程式，你一行都不改
- 二、必做是四段搭建加兩段證明，照表填一定過
- 三、進階親手關掉 **ELB health check**，看服務怎麼壞

前面的動手做，在這裡接起來

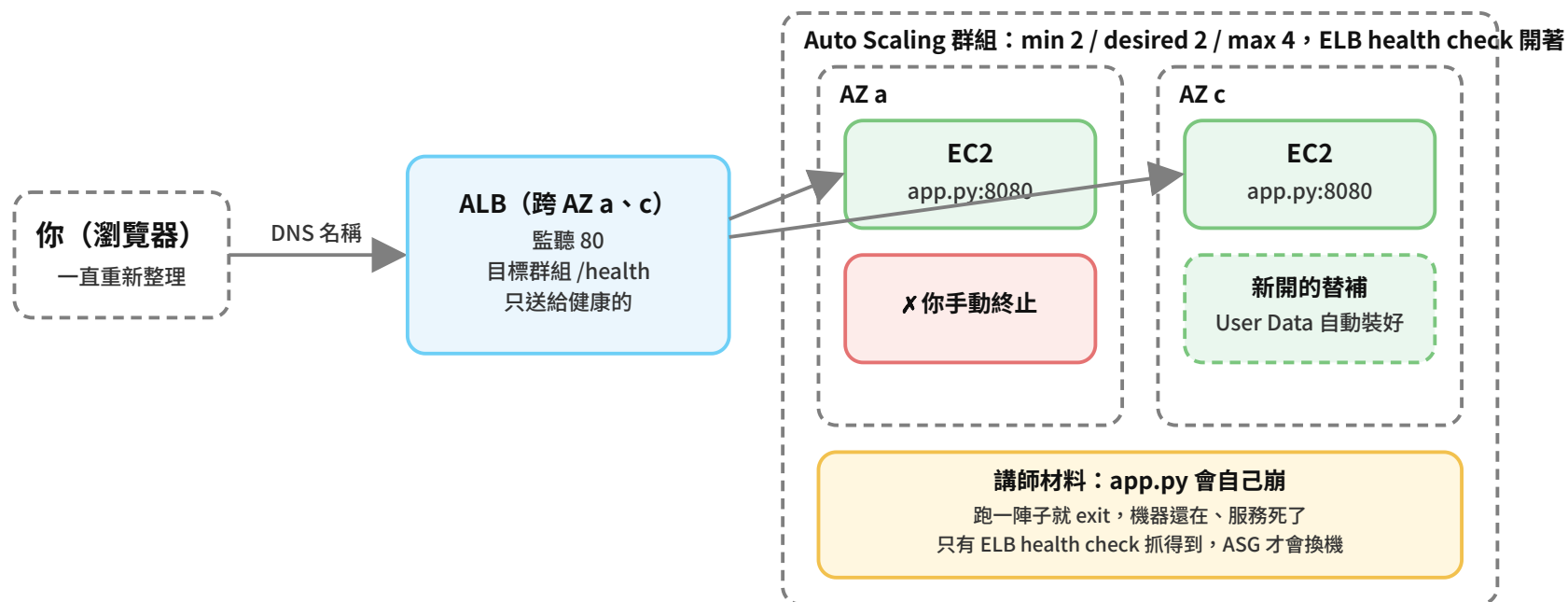
動手做	在 Lab 哪一步再用到
親手「打服務」，再看 health check 的預設值	3-1：路徑從預設的 / 改成 /health，門檻從 5 跟 2 改成 2 跟 2，間隔從 30 改成 10
拆開一個 HTTP 請求，找出規則能看的每一格	—
並排打開 ALB 跟 NLB 的建立頁，找出兩個差別	3-2：這次選 Application Load Balancer，NLB 的三個訊號詞（固定 IP、極高併發、不是 HTTP）一個都沒出現
走一遍 ASG 建立精靈，看三個數字互相卡住	4-1 到 4-5：同一個精靈，這次真的按 Create，三個數字填 2、2、4
打開目標追蹤的指標選單，確認裡面沒有「總量」	加分 7-2：選的就是選單裡的 Application Load Balancer request count per target
讓 curl 扮演閒置逾時 3 秒的 ALB ，看誰被切	—

一句話：前面只看不建的那幾頁，這裡全部真的建一次，看過的欄位直接照填。

驗收：手動終止一台，網站幾十秒內恢復

app.py 聽 8080：/ 回 instance id、AZ、served、will_crash_at，/health 回 200；處理 40–120 個請求後 process 自己結束

實作目標：手動終止任何一台，網站不中斷；程式自己崩潰，也會被換掉



被終止那台手上的請求可能失敗幾秒，記下來就好；這次不要求保住進行中的請求。

- 1 瀏覽器一直重新整理，打的是 ALB 的 DNS 名稱
- 2 紅色那台是你手動終止的，虛線那台是 ASG 自動補上的
- 3 黃框：程式崩了機器還在，幾分鐘內那台要被換掉

只交必做就及格，進階與加分往上疊

三層都用主控台、零程式碼；每個欄位要填什麼都寫在表裡

必做 70%：四段搭建加兩段證明

Security Group、模板、ALB、ASG；再手動終止一台、等程式自己崩。照表填一定過。

進階 20%：關掉考點那一格

親手取消 ELB health check，看目標群組標 unhealthy、ASG 卻不換機。

加分 10%：目標追蹤擴展

狂打請求，讓 desired 從 2 變 3 或 4；縮回不強求。

必做一：機器只收從 ALB 來的流量

搭門口：EC2 → Security Groups → Create，建兩個 Security Group，名稱照填，後面會用到

小步	做什麼	做完應該看到
0	用有管理者權限的身分登入主控台，區域切 Tokyo	全程用預設 VPC，不用自己建網路
1-1	建 alb-sg，VPC 選預設；Inbound 加 HTTP 80，來源 Anywhere-IPv4	alb-sg 入站一條 80
1-2	建 app-sg；Inbound 加 Custom TCP 8080，來源在搜尋框選 alb-sg，不填 IP	來源顯示 sg- 開頭，就是 alb-sg

外面的人繞過 ALB 直接打機器，打不進來；這是 Security Group 的互相參照。

必做二：開每一台都照這張模板

Launch Template 名稱填 `ha-template`；後兩個欄位在 Advanced details 裡

欄位	填什麼	備註
AMI 與機型	Amazon Linux 2023、t3.micro	
Key pair	Don't include in launch template	
Network settings	子網路不選；Security groups 選 app-sg	預設 VPC 的子網路會自動給公有 IP，機器才下載得到程式
IAM instance profile	有 EC2 單元實作留下的 <code>ec2-ssm-role</code> 就選，沒有就留空	方便除錯，不影響驗收
User data	貼 <code>user-data.sh</code> 全文， <code>APP_URL</code> 換成講師給的網址	開機時下載 <code>app.py</code> 並啟動，故意不設自動重啟

做完應該看到：Launch Templates 列表有 `ha-template`，版本 1。

必做三：先建目標群組，再建 ALB

ALB 要指向一個已經存在的目標群組，順序不能反

小步	做什麼	做完應該看到
3-1	建目標群組 ha-tg：Instances、HTTP、Port 8080、VPC 預設；Health check path 填 /health	先別按下一頁，展開 Advanced
3-1 續	Advanced：Healthy threshold 2、Unhealthy threshold 2、Interval 10 秒	下一頁要註冊機器：直接按 Create 不註冊，ASG 之後會放進來
3-2	建 ALB ha-alb：Internet-facing、VPC 預設、Mappings 勾兩個 AZ（例如 1a 跟 1c）	兩個 AZ 各選到一個子網路
3-3	同頁 Security groups 拿掉 default、選 alb-sg；Listener HTTP 80 → Forward to ha-tg；Create	狀態從 Provisioning 變 Active（要幾分鐘）；複製 DNS name

間隔 10 秒、門檻 2 是為了課堂上等得到；正式環境用預設 30 秒也可以。

必做四：勾 **ELB health check**，整堂課的考點

小步	做什麼	做完應該看到
4-1、4-2	建 ha-asg，Launch template 選 ha-template；VPC 預設，子網路選 ALB 那兩個 AZ	各按一次下一頁
4-3	Attach to an existing load balancer → 從 target groups 選 ha-tg	下一頁前先做 4-4
4-4	勾 Turn on Elastic Load Balancing health checks ； grace period 填 60	這一格是整堂課的考點
4-5	Desired 2、Min 2、Max 4；No scaling policies；一路 Next 到 Create	Activity 分頁出現兩筆 Launching
4-6	等三四分鐘，到 ha-tg 的 Targets 分頁	兩台都是 healthy

grace period 60 秒留给 User Data 下載跟啟動；設太短會**反覆換機**，常見的第一個坑。

實作：必做五，終止一台再等它自己崩

兩段證明；記錄讓它一直跑。Windows 沒有這行指令，就一直按重新整理

1

瀏覽器打 **ha-alb** 的 **DNS name (http)**，重新整理幾次

做完應該看到：instance 在兩台之間跳

2

開始記錄：Mac 或 Linux 在終端機跑這一行

```
$ while true; do echo "$(date +%T) $(curl -s -m 2 http://你的ALB網址/ || echo 失敗)"; sleep 1; done
```

做完應該看到：每秒一行，有時間跟回應

3

證明一：Instances 挑一台 → Instance state → Terminate，再到 **ha-asg** 的 **Activity** 分頁

記錄出現幾行「失敗」後只剩另一台回應；一兩分鐘內出現新的 Launching，幾分鐘後 **ha-tg** 又是兩台 healthy

4

證明二：什麼都不做，看 **served** 快到 **will_crash_at** 的那台

崩了之後 **ha-tg** 標 unhealthy，ASG 幾分鐘內終止它、補一台；Activity 那筆的原因寫到 ELB health check

進階：關掉那一格，壞機器沒人換

兩層健康檢查各看各的：負載平衡器知道服務死了，要勾那一格 ASG 才會知道

小步	做什麼	做完應該看到
6-1	ha-asg → Details → Health checks → Edit，取消勾選 Elastic Load Balancing health checks，存檔	設定只剩 EC2
6-2	等一台程式崩掉	ha-tg 標 unhealthy，但 Activity 沒有新的一筆
6-3	把勾選打開	幾分鐘內那台被換掉

ALB 不再送流量過去，但機器還活著，ASG 不換；只剩一台在撐，再崩一台就全掛。

加分做目標追蹤，清理每個人都要做

小步	做什麼	做完應該看到
7-1	發新版 Launch Template，User Data 的 FLAKY_NO_CRASH 改成 1；ASG 指到新版本；兩台舊機各終止一次	新機器不會再崩
7-2	ha-asg → Automatic scaling 加 Target tracking：指標 ALB request count per target，目標值 30	規則列表有一條
7-3	把記錄那行的 sleep 1 拿掉，狂打幾分鐘	desired 變 3 或 4，Activity 有 scale out；停十幾分鐘後縮回 2
清理	Desired 跟 Min 改 0，等機器都 Terminated；再依序刪 ha-asg、ha-alb、ha-tg、ha-template、app-sg、alb-sg	順序跟搭建相反；ec2-ssm-role 可以留著

不改成 1，狂打會先觸發崩潰；ALB 跟 EC2 開著每小時都在算錢，ASG 本身不收費。

卡住先看 **Health status details** 寫什麼

卡在哪	看到什麼	怎麼辦
4-6 unhealthy	ha-tg Targets 分頁寫 Request timed out	多半是 app-sg 的 8080 來源沒選 alb-sg
4-6 unhealthy	寫 Health checks failed with these codes: [404]	health check 路徑沒填 /health
4-6 機器一直被換	User Data 的 APP_URL 沒換掉，程式根本沒起來	發新版 Launch Template，ASG 指到新版本，再終止舊機器
3-3 回 503	還沒有任何 healthy 的機器	先解決 4-6
想看機器裡面	有掛 ec2-ssm-role 才進得去	用 Session Manager 進機器看 /var/log/flaky.log

Health status、Activity、log 這三個地方，就是之後**可觀測性**的雛形。

個人作業：三張截圖加三行

這個實作就是作業，個人完成；只交必做就及格

- 1 截圖三張：ha-tg 兩台 healthy；手動終止時的記錄加 ASG Activity；程式崩後的 Activity
- 2 三行：終止到補齊幾分幾秒、崩潰到換掉幾分幾秒、進階那段看到什麼（沒做就寫「沒做」）
- 3 做了加分的，附 scale out 那一筆 Activity
- 4 評分：必做 70%（截圖各 15%、兩個時間各 12.5%）、進階 20%、加分 10%

驗收 進階要寫出「目標群組 unhealthy 但 ASG 不換機」才算對；加分 desired 上升就給，縮回不強求。

回家作業：六份都是個人完成

每份都拿你的期末專題來寫

單點失敗與健康檢查

替專題 API 寫 `/health` 規格，五到八行；再寫哪種故障它抓不到。

ALB 第七層路由

一張 ALB 規則表，至少三條含預設；加一行 80 埠怎麼處理。

NLB 第四層場景

替三個服務各選 ALB 或 NLB，各寫一行理由。

ASG 基礎

專題 agent API 的 ASG 規格，八到十行。

擴展策略

一條目標追蹤策略：指標、目標值、理由、warmup，共五行。

LLM 應用的擴展

用三個會壞的地方檢查上一份：原本、問題、改成，三列三欄。

TAKEAWAY

- 01** 機器活著不代表服務活著；要問服務就得打服務。
- 02** ALB 看信的內容，NLB 只看信封；要快、要固定 IP、非 HTTP 才用 NLB。
- 03** ASG 補到期望台數；勾 ELB health check 才看得到服務死，指標看每台多忙。
- 04** LLM 服務看併發不看 CPU、連線要撐得住、健康檢查要真的問一句。

Reference

- **EC2 User Guide : Status checks for Amazon EC2 instances** [三種掛法](#)
- **ELB User Guide : How Elastic Load Balancing works** [負載平衡器三件事](#)
- **ALB Guide : Health checks for target groups** [health check 怎麼判斷](#)
- **ALB Guide : Condition types for listener rules** [規則的條件與動作](#)
- **NLB Guide : What is a Network Load Balancer** [第四層與固定 IP](#)
- **EC2 Auto Scaling User Guide : About the health checks** [自癒與 ELB health check](#)
- **EC2 Auto Scaling User Guide : Target tracking scaling policies** [選指標的規則](#)
- **ALB Guide : Edit attributes** [閒置逾時](#)